

NAME: COLLINS AMADI
STUDENT ID: FWSD2511357
STUDENT EMAIL: fwsd2511357@students.icdfa.edu.ng
COURSE CODE: SBT-DF203
LAB NAME: SYN FLOOD PATTERN INVESTIGATION USING TSHARK
DATE: 11/09/2026
INSTRUCTOR: AMINU IDRIS

Important Pre-Lab Instruction

Read the matching lecture slide deck before executing the practical. Work only inside the ICDFA-approved lab or your own isolated virtual environment. Record every command, observation and screenshot as contemporaneous evidence.

Course Code	SBT-DF203
Course Title	Basic Networking Skills for Digital Forensics
Lab Number	Lab 3
Lab Title	SYN Flood Attack Investigation Using tshark
Delivery Block	1/3 of 3
Scheduled Dates	5-11 September 2026
Estimated Duration	3-4 hours practical work plus report writing
Mode	Individual practical lab in an authorized virtual environment
Required Evidence	mySYNFloodCapture.pcap or a bounded loopback capture generated in the lab

1. Source Materials and Slide Alignment

- HTTP, tshark and SYN Flood: TCP three-way handshake, half-open connections, SYN flood concept, Scapy packet generation and tshark capture.
- The uploaded slide uses Apache on port 80, loopback capture and a four-packet controlled simulation.
- The assignment requires two authorized VMs or an equivalent isolated environment and evidence that distinguishes attack-like SYN activity from a normal handshake.

Reading Requirement Before Lab Execution

The lab deliberately follows the sequence, terminology and core exercises presented in the uploaded SBT-DF203 slides. Commands have been corrected where needed for Linux syntax and forensic repeatability.

2. Lab Scenario

An Apache service experienced repeated TCP SYN packets that were not followed by the final ACK of the three-way handshake. You will preserve the packet capture, establish a normal-traffic baseline, identify suspicious SYN patterns, calculate indicators, and conduct a strictly bounded loopback simulation to validate your interpretation.

3. Learning Outcomes

- Differentiate a complete TCP handshake from an incomplete or half-open attempt.
- Use tshark display filters and field extraction to identify SYN, SYN-ACK, ACK and reset packets.
- Calculate SYN counts, unique source ports, response behaviour and incomplete-handshake indicators.
- Create a four-packet loopback-only simulation with Scapy in an authorized environment.
- Develop a defensible timeline and incident finding from PCAP evidence.
- Recommend controls for detecting and mitigating SYN flood activity.

4. Legal, Ethical and Safety Rules

- Use only systems, packet captures and networks for which you have explicit authorization.
- Keep the lab isolated from production, campus, office and public networks. Use NAT or host-only virtual networking as instructed.
- Do not collect, disclose or reuse real credentials, personal messages or confidential traffic.

- Preserve original capture files. Perform analysis on verified working copies and record SHA-256 hashes.
- Stop any traffic-generation or interception process immediately after the required evidence is captured.
- Restore firewall, ARP, forwarding and network settings before closing the lab.
- The simulation is limited to four SYN packets against 127.0.0.1 or an instructor-assigned private lab IP. Do not increase packet count, automate continuous sending, spoof third-party addresses or target public systems.

5. Required Tools and Environment

Item	Recommended Tool	Purpose
Operating system	Kali Linux VM	Run tshark, Apache and Scapy.
Target service	Local Apache on 127.0.0.1:80	Safe target for bounded simulation.
Capture tool	tshark/Wireshark	Capture and analyze TCP flags.
Packet library	python3-scapy	Generate a small number of SYN packets.
Evidence	Provided PCAP or student capture	Forensic analysis source.
Optional second VM	Authorized host-only target	Demonstrate network capture without touching external networks.

6. Lab Folder Structure and Evidence Preparation

```
mkdir -p ~/SBT-DF203-Lab3/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab3
pwd
find . -maxdepth 1 -type d -print
```

Create the folder structure before downloading or generating evidence. Store original captures under evidence and analysis copies under working.

```
sudo apt update
sudo apt install -y apache2 tshark wireshark python3-scapy
sudo systemctl enable --now apache2
# Optional supplied capture
wget -O evidence/mySYNFloodCapture.pcap \
  'https://raw.githubusercontent.com/frankwxu/digital-forensics-
  lab/main/Illegal_Possession_Images/lab_files/SYN_Flood/mySYNFloodCapture.pcap'
sha256sum evidence/mySYNFloodCapture.pcap | tee reports/syn_capture_sha256.txt
```

Screenshot required: The Apache service state, evidence file details and SHA-256 hash.

7. Mini Evidence and Chain-of-Custody Worksheet

ANSWER:

Field	Student Entry
Case/lab identifier	SBT-DF203-Lab3-COLLINS AMADI
Trainee name	COLLINS AMADI
Date and time started	11/09/2026 05:04:21
Evidence file name(s)	evidence/mysynfloodcapture.pcap
Source or generation method	KALI LINUX COMMAND
Original SHA-256	14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5 evidence/mySYNFloodCapture.pcap
Working-copy SHA-256	working/bounded_syn_activity_working.pcapng
Analysis workstation/VM	ROOT@KALI
Notes on any changes	Original preserved read-only in evidence folder. Timestamp-preserved working copy created for analysis. No packet modification performed

8. Part A - Establish a Normal Handshake Baseline

```
sudo tshark -i lo -f 'tcp port 80' -a duration:15 -w evidence/normal_http.pcapng &
sleep 2
curl --no-keepalive http://127.0.0.1/ >/dev/null
wait

tshark -r evidence/normal_http.pcapng -Y 'tcp.flags.syn==1 || tcp.flags.fin==1' -T fields \
  -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
  | tee reports/normal_handshake_flags.tsv
```

Document the expected SYN, SYN-ACK and ACK sequence before examining suspicious traffic.

9. Part B - Conduct the Bounded Loopback Simulation

Create the following script exactly. It sends four SYN packets to the local Apache service and then exits. It does not run continuously and does not target another organization.

```
cat > scripts/syn_probe_lab.py <<'PYLAB'
from scapy.all import IP, TCP, RandShort, send

TARGET = '127.0.0.1'
PORT = 80
COUNT = 4

packets = [IP(dst=TARGET)/TCP(sport=RandShort(), dport=PORT, flags='S') for _ in range(COUNT)]
send(packets, verbose=False)
print(f'Sent {COUNT} authorized training SYN packets to {TARGET}:{PORT}')
PYLAB

# Terminal 1
sudo tshark -i lo -f 'tcp port 80' -c 20 -w evidence/bounded_syn_activity.pcapng

# Terminal 2, while capture is running
sudo python3 scripts/syn_probe_lab.py
```

Stop Condition

If the capture reaches 20 packets it stops automatically. Otherwise stop with Ctrl+C immediately after the four SYN packets and associated responses are captured.

10. Part C - Identify SYN Indicators with tshark

```
PCAP=working/bounded_syn_activity_working.pcapng
cp --preserve=timestamps evidence/bounded_syn_activity.pcapng "$PCAP"
sha256sum evidence/bounded_syn_activity.pcapng "$PCAP" | tee reports/bounded_capture_hashes.txt

# Initial SYNs only
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.seq \
  | tee reports/initial_syns.tsv

# SYN-ACK responses
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==1' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.ack \
  | tee reports/syn_ack_responses.tsv

# Resets and final ACK candidates
tshark -r "$PCAP" -Y 'tcp.flags.reset==1 || (tcp.flags.ack==1 && tcp.len==0)' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
  | tee reports/ack_reset_candidates.tsv
```

11. Part D - Quantify and Correlate the Activity

```
# Counts by source and destination
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
  -e ip.src -e ip.dst -e tcp.dstport | sort | uniq -c | sort -nr \
  | tee reports/syn_counts_by_pair.txt

# Unique client source ports
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields -e tcp.srcport \
  | sort -n | uniq | tee reports/unique_syn_source_ports.txt

# Expert information and TCP analysis flags
tshark -r "$PCAP" -q -z expert | tee reports/expert_info.txt
```

```
tshark -r "$PCAP" -Y 'tcp.analysis.retransmission || tcp.analysis.lost_segment || tcp.analysis.duplicate_ack' \
-T fields -e frame.number -e frame.time -e _ws.col.Info | tee reports/tcp_analysis_events.tsv
```

12. Part E - Compare Normal and Suspicious Sessions

ANSWER:

Indicator	Normal HTTP Session	Bounded SYN Activity	Forensic Meaning
Initial SYN count	Initial SYNs1	connection attempt	4 attempts
SYN-ACK count	SYN-ACKs	1	About 4
Completed handshakes	Completed handshakes	1	0
Unique client source ports	Unique source ports	Usually 1	4 random ports
HTTP request present?	HTTP request present	Yes	No
RST packets	RST packets	Usually none	Often 4
Observed duration	Meaning	Legitimate session	Half-open, SYN-flood-like pattern

A real SYN flood is defined by scale, rate, persistence and service impact. Four training packets are not a denial-of-service event; they reproduce the packet pattern in a safe way so that analysts can learn the indicators.

```
(root@kali)~[/SBT-DF203-Lab3]
# mkdir -p ~/SBT-DF203-Lab3/evidence,working,exported,reports,screenshots,scripts
# cd ~/SBT-DF203-Lab3
# pwd
/root/SBT-DF203-Lab3
# find . -maxdepth 1 -type d -print
./evidence
./screenshots
./scripts
./exported
./reports
./working
# sudo apt update
Hit:1 https://http.kali.org/kali kali-rolling InRelease
1254 packages can be upgraded. Run 'apt list --upgradable' to see them.
Warning: Problem unlinking the file prtial - pkgAcquire::Clean (21: Is a directory)

# sudo apt install -y apache2 tshark wireshark python3-scapy
apache2 is already the newest version (2.4.68-1).
tshark is already the newest version (4.6.6-1).
wireshark is already the newest version (4.6.6-1).
python3-scapy is already the newest version (2.7.0+dfsg1-1).
Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1254

# sudo systemctl enable --now apache2

# wget -O evidence/mySYNFloodCapture.pcap \
'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/lab_files
/SYN_Flood/mySYNFloodCapture.pcap'
Prepended http:// to '
--2026-09-11 20:38:31-- http://%/
Resolving ( )... failed: Name or service not known.
wget: unable to resolve host address '
--2026-09-11 20:38:31-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Posse
ssion_Images/lab_files/SYN_Flood/mySYNFloodCapture.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.111.133, 185.199
.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443 ... connected.
```

```
Connecting to raw.githubusercontent.com (raw.githubusercontent.com) [185.199.108.133]:443... Connected.  
HTTP request sent, awaiting response... 200 OK  
Length: 260716 (255K) [application/octet-stream]  
Saving to: 'evidence/mySYNFloodCapture.pcap'  
evidence/mySYNFloodCapture 100%[=====] 254.61K 109KB/s in 2.3s  
2026-09-11 20:38:37 (109 KB/s) - 'evidence/mySYNFloodCapture.pcap' saved [260716/260716]  
FINISHED --2026-09-11 20:38:37--  
Total wall clock time: 5.7s  
Downloaded: 1 files, 255K in 2.3s (109 KB/s)
```

```
(root@kali)-[~/SBT-DF203-Lab3]  
# sha256sum evidence/mySYNFloodCapture.pcap | tee reports/syn_capture_sha256.txt  
14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5 evidence/mySYNFloodCapture.pcap
```

```
(root@kali)-[~/SBT-DF203-Lab3]  
# sudo tshark -i lo -f 'tcp port 80' -a duration:15 -w evidence/normal_http.pcapng &  
[1] 37585
```

```
(root@kali)-[~/SBT-DF203-Lab3]  
# Running as user "root" and group "root". This could be dangerous.  
Capturing on 'Loopback: lo'
```

Session Actions Edit View Help

```
[1] + done sudo tshark -i lo -f 'tcp port 80' -a duration:15 -w  
(root@kali)-[~/SBT-DF203-Lab3]  
# sleep 2
```

```
(root@kali)-[~/SBT-DF203-Lab3]  
# curl --no-keepalive http://127.0.0.1/ >/dev/null  
% Total % Received % Xferd Average Speed Time Time Current  
Dload Upload Total Spent Left Speed  
100 1578 100 1578 0 0 340.9k 0 0 0 0
```

```
(root@kali)-[~/SBT-DF203-Lab3]  
# wait
```

```
(root@kali)-[~/SBT-DF203-Lab3]  
# tshark -r evidence/normal_http.pcapng -Y 'tcp.flags.syn==1 || tcp.flags.fin==1' -T fields \  
-e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \  
| tee reports/normal_handshake_flags.tsv  
Running as user "root" and group "root". This could be dangerous.  
tshark: "-Tfields" was specified, but no fields were specified with "-e".
```

```
(root@kali)-[~/SBT-DF203-Lab3]  
# cat > scripts/syn_probe_lab.py <<'PYLAB'  
from scapy.all import IP, TCP, RandShort, send
```

```
TARGET = '127.0.0.1'  
PORT = 80  
COUNT = 4
```

```
packets = [IP(dst=TARGET)/TCP(sport=RandShort(), dport=PORT, flags='S') for _ in range(COUNT)]  
send(packets, verbose=False)  
print(f'Sent {COUNT} authorized training SYN packets to {TARGET}:{PORT}.')  
PYLAB
```

```
(root@kali)-[~/SBT-DF203-Lab3]  
# sudo tshark -i lo -f 'tcp port 80' -c 20 -w evidence/bounded_syn_activity.pcapng  
Running as user "root" and group "root". This could be dangerous.  
Capturing on 'Loopback: lo'
```

^C

```
(root@kali)-[~/SBT-DF203-Lab3]  
# sudo python3 scripts/syn_probe_lab.py  
Sent 4 authorized training SYN packets to 127.0.0.1:80
```

```
(root@kali)-[~/SBT-DF203-Lab3]  
# PCAP=working/bounded_syn_activity_working.pcapng
```

```
(root@kali)-[~/SBT-DF203-Lab3]  
# PCAP=working/bounded_syn_activity_working.pcapng
```

```
(root@kali)-[~/SBT-DF203-Lab3]  
# cp --preserve=timestamps evidence/bounded_syn_activity.pcapng "$PCAP"
```

```
(root@kali)-[~/SBT-DF203-Lab3]  
# sha256sum evidence/bounded_syn_activity.pcapng "$PCAP" | tee reports/bounded_capture_hashes.txt  
b96f324cfa85ebb6ebd0a2f24e04de579632bef0fa5dd85ee71b60dd08e07638 evidence/bounded_syn_activity.pcapng  
b96f324cfa85ebb6ebd0a2f24e04de579632bef0fa5dd85ee71b60dd08e07638 working/bounded_syn_activity_working.pcapng
```

```
(root@kali)-[~/SBT-DF203-Lab3]  
# tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \  
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.seq \  
| tee reports/initial_syns.tsv  
Running as user "root" and group "root". This could be dangerous.  
tshark: "-Tfields" was specified, but no fields were specified with "-e".
```

```
(root@kali)-[~/SBT-DF203-Lab3]  
# tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==1' -T fields \  
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.ack \  
| tee reports/syn_ack_responses.tsv
```

```

(root@kali)~/SBT-DF203-Lab3
# PCAP=working/bounded_syn_activity_working.pcapng

(root@kali)~/SBT-DF203-Lab3
# cp --preserve=timestamps evidence/bounded_syn_activity.pcapng "$PCAP"

(root@kali)~/SBT-DF203-Lab3
# sha256sum evidence/bounded_syn_activity.pcapng "$PCAP" | tee reports/bounded_capture_hashes.txt
b96f324cfa85ebb6ebd0a2f24e04de579632bef0fa5dd85ee71b60dd08e07638 evidence/bounded_syn_activity.pcapng
b96f324cfa85ebb6ebd0a2f24e04de579632bef0fa5dd85ee71b60dd08e07638 working/bounded_syn_activity_working.pcapng

(root@kali)~/SBT-DF203-Lab3
# tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.seq \
| tee reports/initial_syns.tsv
Running as user "root" and group "root". This could be dangerous.
tshark: "-Tfields" was specified, but no fields were specified with "-e".

(root@kali)~/SBT-DF203-Lab3
# tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==1' -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.ack \
| tee reports/syn_ack_responses.tsv
Running as user "root" and group "root". This could be dangerous.
tshark: "-Tfields" was specified, but no fields were specified with "-e".

(root@kali)~/SBT-DF203-Lab3
# tshark -r "$PCAP" -Y 'tcp.flags.reset==1 || (tcp.flags.ack==1 && tcp.len==0)' -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
| tee reports/ack_reset_candidates.tsv
Running as user "root" and group "root". This could be dangerous.
tshark: "-Tfields" was specified, but no fields were specified with "-e".

(root@kali)~/SBT-DF203-Lab3
# tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
-e ip.src -e ip.dst -e tcp.dstport | sort | uniq -c | sort -nr \
| tee reports/syn_counts_by_pair.txt
sort: cannot read: : No such file or directory
Running as user "root" and group "root". This could be dangerous.
tshark: "-Tfields" was specified, but no fields were specified with "-e".

(root@kali)~/SBT-DF203-Lab3
# tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields -e tcp.srcport \
| sort -n | uniq | tee reports/unique_syn_source_ports.txt
Running as user "root" and group "root". This could be dangerous.
tshark: Display filters were specified both with "-Y" and with additional command-line arguments.

(root@kali)~/SBT-DF203-Lab3
# tshark -r "$PCAP" -q -z expert | tee reports/expert_info.txt
Running as user "root" and group "root". This could be dangerous.

(root@kali)~/SBT-DF203-Lab3
# tshark -r "$PCAP" -Y 'tcp.analysis.retransmission || tcp.analysis.lost_segment || tcp.analysis.duplicate_ack' \
-T fields -e frame.number -e frame.time -e _ws.col.Info | tee reports/tcp_analysis_events.tsv
Running as user "root" and group "root". This could be dangerous.
tshark: Display filters were specified both with "-Y" and with additional command-line arguments.

(root@kali)~/SBT-DF203-Lab3
#

```

direct input to this VM, move the mouse pointer inside or press Ctrl+G.



13. Detection and Mitigation Recommendations

- Alert on elevated SYN-to-completed-handshake ratios per source and destination.
- Monitor SYN backlog exhaustion, retransmissions, connection-state tables and service latency.
- Use SYN cookies, sensible backlog/time-out tuning, upstream rate limiting and DDoS protection where appropriate.
- Retain full packet capture or flow telemetry during incidents and synchronize system clocks.
- Correlate packet evidence with web server, firewall, load balancer and operating-system logs.

ANSWER:

Enable SYN cookies, tune SYN backlog and timeouts, rate-limit excessive SYNs, monitor SYN-to-completed-handshake ratios, retain PCAP/flow logs, and correlate with Apache, firewall, load-balancer, and OS logs.